



DocPulse

PORTAL

Doctor Appointment Platform

TECHNICAL OVERVIEW

Architecture, Data Model & Product Walkthrough

Prepared September 2026



One platform, three roles, one shared calendar.

DocPulse connects doctors and patients on a single platform: a clinic (or single admin) onboards doctors and patients by invitation, doctors publish blocks of time they're available, and patients search, book, and track appointments inside those windows — all backed by a small, explicit state machine and database-level double-booking prevention.



Admin

GATEKEEPER

Invites doctors and other admins by email (one at a time, or via bulk CSV), manages the invitation list, and revokes access.



Doctor

PROVIDER

Publishes availability windows, reviews incoming requests, and confirms, rejects, or completes appointments.



Patient

SELF-SERVICE

Self-registers or accepts an invite, searches doctors by specialization, and books, tracks, or cancels appointments.

A focused, conventional stack—no framework sprawl.

Every layer picks one well-understood tool and uses it directly: no state-management library, no ORM abstraction beyond TypeORM, no microservices.

FRONTEND

React 19 — client-rendered SPA

TypeScript — end to end

Vite 8 — dev server & bundler

Tailwind CSS 4 — utility styling

Custom router — no react-router

React Context — Auth + Router state

Vitest + RTL + MSW — component tests

BACKEND

Node.js + Express 4 — REST API

TypeORM — Postgres ORM, snake_case

JWT + bcrypt — stateless auth

Joi — request validation

Helmet · CORS · rate-limit — security

Winston — structured logging

Jest + Supertest — integration tests

DATA & INFRA

PostgreSQL — tstzrange + GIST

Nodemailer — transactional email

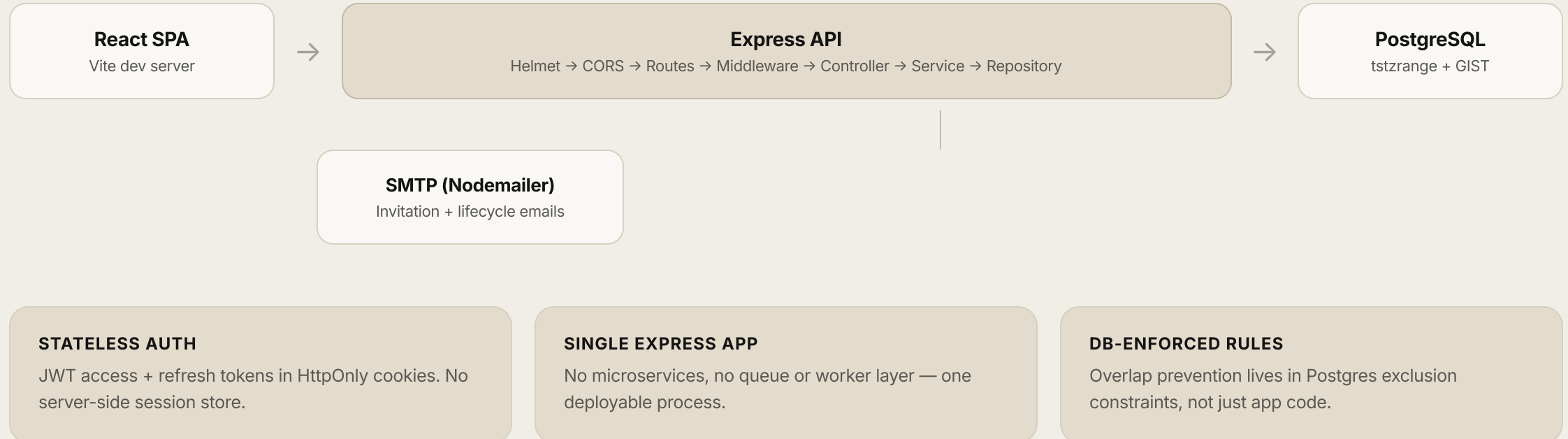
Swagger / OpenAPI — live API docs

Docker Compose — full-stack local run

Sentry — error monitoring

Playwright — end-to-end suite

Request flow, end to end.



Stateless JWT, defense in depth.

LOGIN & SILENT REFRESH

- 1 POST /auth/login → bcrypt.compare
- 2 Access (15m) + refresh (7d) tokens set as HttpOnly cookies
- 3 Any request → 401 → apiFetch calls /auth/refresh once
- 4 New access token issued → original request retried
- 5 Refresh fails → logout → session-expired event

DEFENSES IN PLACE

-  **Helmet** security headers on every response, unconditionally.
-  **CORS** locked to a single configured FRONTEND_URL origin.
-  **4 rate limiters** — general, auth, invitation, self-registration.
-  **IDOR-safe queries** — ownership filtered in the SQL, not after the fact.

Two ways in: invited, or self-registered.

Every account still resolves through the same accept-invitation endpoint — self-registration just changes who creates the invitation row.

Admin-invited

EXISTING

- 1 Admin sends invite (email + role)
- 2 Race-proof insert, partial-unique-index backed
- 3 Invitation email with a 24h token link
- 4 Invitee accepts → sets password → account created

Patient self-registration

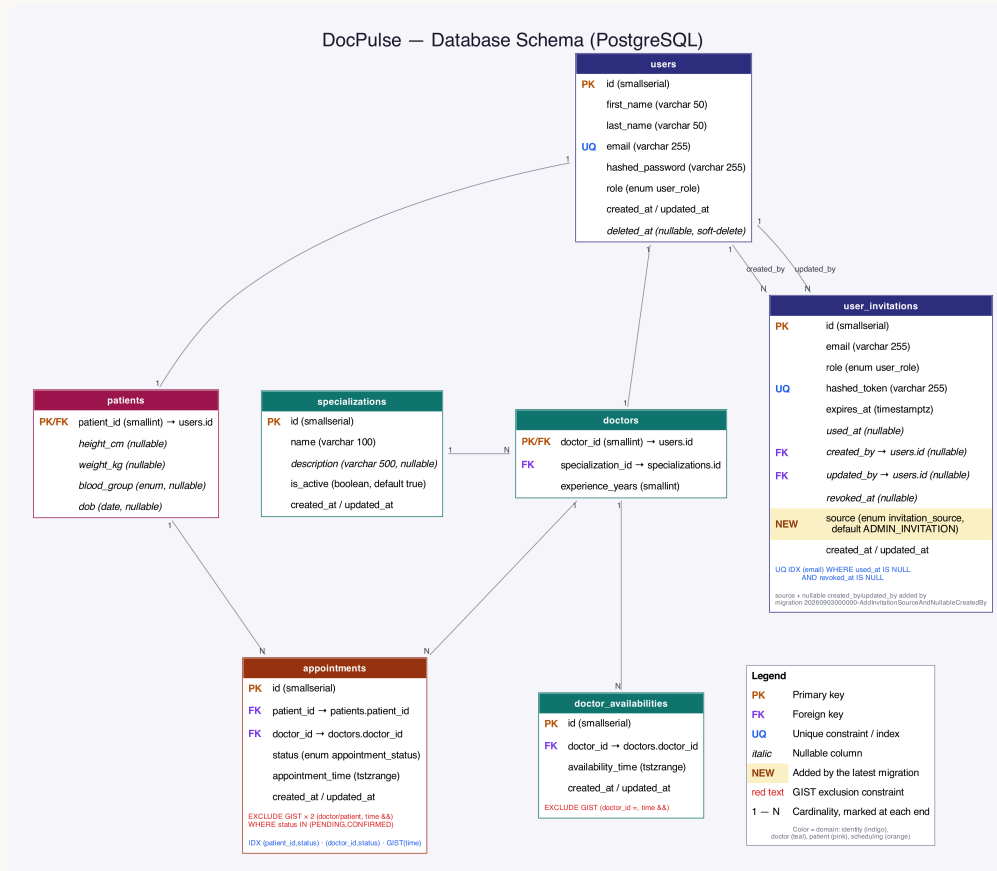
NEW

- 1 Patient requests a signup link with their email
- 2 Enumeration-safe response either way
- 3 Invitation row created with source = PATIENT_SELF_REGISTRATION, no inviting admin
- 4 Same accept-invitation flow completes signup



Schema change: `user_invitations.source` (enum `invitation_source`) was added, and `created_by` / `updated_by` became nullable — a self-requested invitation has no inviting admin. See Data Model, next.

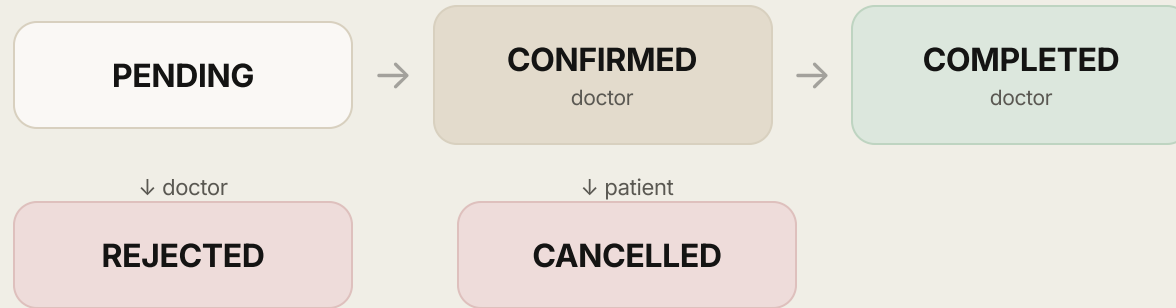
7 tables, PostgreSQL-enforced correctness.



NEW — SOURCE (INVITATION_SOURCE ENUM)

Double-booking is blocked by GIST exclusion constraints — a database guarantee, not just a service-layer check.

A small, explicit state machine.



TIMING GUARDS

Can't confirm a past appointment, can't complete before it starts, can't cancel once the start time has passed — all anchored to IST wall-clock time.

STALE-PENDING AUTO-EXPIRY

A **PENDING** request left untouched for 48h is lazily swept to **REJECTED** on the next listing call.

BOOKING-ABUSE CAPS

Max 2 active requests per doctor, 5 active total per patient.

Free-slot computation, conflict-proof booking.

- 1 Doctor publishes an availability window — stored as a Postgres `tstzrange`, exclusion-constrained against their own overlapping windows.
- 2 Patient requests a doctor's **bookable free time** — raw availability, minus anything already covered by that doctor's PENDING/CONFIRMED appointments.
- 3 Patient books a slot fully contained within one published window (@> range containment check).
- 4 **Race protection:** a GIST exclusion constraint on `appointment_time` rejects any overlapping insert — even under concurrent requests — independent of the application-level check above.

2

GIST exclusion constraints on appointments

30m

Booking sub-slot granularity in the UI

0

Double-bookings possible by design

One shared component system, six lifecycle emails.

SHARED COMPONENTS

shell — outer layout wrapper

header · footer

infoCard · divider · button

Every template composes the same building blocks, so tone and layout stay consistent across the whole lifecycle.

EMAILS SENT

- Invitation (admin or self-registration)
- Appointment requested
- Appointment confirmed
- Appointment declined
- Appointment cancelled
- Appointment completed

Failure isolation: an SMTP failure on invitation deletes the orphaned invite; on self-registration it fails silently (enumeration-safe); on appointment lifecycle events it's logged only — the underlying operation still succeeds either way.

Real Postgres, no mocking.

Every layer of the stack has a dedicated, real-dependency test suite rather than stubbed-out unit tests alone.



Backend

Jest + Supertest against the real Express app and a real disposable Postgres database — truncated and reseeded between tests, not mocked.



Frontend

~30 Vitest + React Testing Library files across pages, components, contexts and API wrappers, with network calls intercepted by MSW.



End-to-end

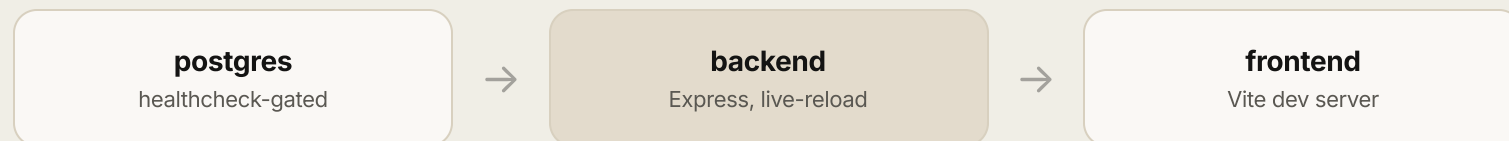
Playwright drives a real browser against the real backend and frontend together — booking, conflicts, onboarding, cancellations.

Known issues, already scoped for a fix.

From the latest dependency audit — ordered by severity, with a fix path identified for each.

TypeORM ^0.2.29 · direct, prod	CRITICAL	SQL injection (CVSS 9.8) + 2 more SQLi advisories, 1 code-injection. No auto-fix — needs the 0.2→0.3.x major (breaking Repository/QueryBuilder API), blocked by typeorm-pagination — an apparently-abandoned direct dependency that bundles mocha as a runtime dep and must be replaced with hand-rolled pagination first.
axios 0.21.1 · direct, prod	HIGH	20+ advisories — CSRF, SSRF, prototype pollution, ReDoS, credential/header leakage. Fix: major bump to 1.20.0+.
nodemailer 6.10.1 · direct, prod	HIGH	SMTP/header injection, SSRF via the raw option, arbitrary file read. Fix: major bump to 9.1.1.
express / body-parser 4.17.1 / 1.19.0 · direct, prod	HIGH	DoS, open redirect, XSS via redirect. Fix available without a major bump.
aws-sdk ^2.818.0 · direct	MODERATE	Low-severity CVE directly, but v2 is deprecated / maintenance-mode per AWS. Longer-term: migrate to AWS SDK v3 (@aws-sdk/client-*).
npx ^3.0.0 · unused devDependency	HOUSEKEEPING	Redundant — npm has shipped npx as a CLI command since npm 5.2 (2017). Drags in ~15 transitive vulnerabilities alone, including 2 of the 6 criticals (rc→deep-extend, minimist) plus execa/cross-spawn/got/y18n/yargs. One-line removal.
sha.js / tar transitive, build-time	BUILD-TIME	Pulled in via @mapbox/node-pre-gyp (bcrypt's native build step). No runtime/code impact — auto-fixable with npm audit fix.

One command, the whole stack.



```
$ docker compose up --build  
$ docker compose exec backend npm run migrate
```

One bridge network, one Postgres volume for persistence, and the same ports as local dev.



Also shareable live via ngrok — the same ports mean an existing tunnel to the frontend keeps working unchanged, letting the team preview a running instance without a deployment.



DocPulse

Built with React, Express & PostgreSQL.